BND Reporting Platform

Project Scope & Technical Proposal

Read-Only Benefits & Billing Reporting Portal

Proposed Core Stack

Frontend: Next.js | Backend: FastAPI / Python | Reporting Database: PostgreSQL

Prepared from the supplied prototype, Sage 100 sample database extract, and confirmed business requirements.

1. Executive Summary

The proposed solution is a modern, branded, read-only reporting portal that gives end users, location managers, clients, brokers, and internal administrators secure access to the records and reports they are authorized to view.

Sage 100 remains the authoritative source of business data. The portal will not write transactional data back to Sage. Instead, selected source data will be synchronized into a PostgreSQL reporting database and exposed through a FastAPI API to a responsive Next.js web application.

Key Architecture Principle

Sage/ERP data always wins. The web portal is a reporting layer, not a replacement transaction system.

2. Project Objectives

Provide secure, role-aware access to benefits, billing, employee/member, invoice, and related reporting data.

Support a hierarchical access model from Location to Client to Broker to internal ALL access.

Allow end users to self-register with email/phone verification and gain access only to their own matched records.

Automate transfer/synchronization of required Sage data into a reporting database.

Support datasets that grow into the multi-million-record range without loading full tables into the browser.

Keep the solution independent enough to support a future move from Sage 100 to Sage 100 on SQL Server or Odoo.

Deliver a branded web experience that can be linked from or presented under the existing company website/domain.

3. Confirmed Business & Technical Requirements

Area	Confirmed Requirement
Source of truth	Sage 100 / ERP data is authoritative.
Application behavior	Read-only for ERP business data.
Source tables	Approximately 10 production source tables are expected.
Data volume	Largest table is approximately 800,000 records today and may grow to approximately 2,000,000 records.
Synchronization	Preferred: live where practical, otherwise automated daily synchronization.
Freshness indicator	Users should see the last successful update date/time.
End-user access	Self-registration with lightweight email and phone verification; own records only.
Manager/Admin access	Pre-approved hierarchical access.
Hierarchy	Location → Client → Broker → ALL.
Branding/hosting	Should appear as part of the existing company web presence; separate hosting/subdomain is acceptable if needed.

4. Proposed Technology Stack

Next.js (Frontend)

Responsive reporting portal, dashboards, end-user views, administrative workflows, and branded web experience.

FastAPI / Python (Backend)

Authentication, hierarchical authorization, reporting APIs, synchronization orchestration, exports, audit logging, and business access rules.

PostgreSQL (Reporting Database)

Optimized reporting datastore, indexing, server-side filtering, multi-million-record scalability, and standard ODBC compatibility.

Why a Separate Reporting Database?

It isolates reporting traffic from the ERP, allows controlled indexing and aggregation, improves web performance, and reduces future migration impact if the source ERP changes.

5. Proposed Architecture

Layer	Purpose
1. Source System	Sage 100 today; potentially Sage 100 + Microsoft SQL Server or Odoo in the future.
2. Integration Layer	ODBC, direct SQL access, scheduled exports, or another supported source connector depending on final Sage configuration.
3. Reporting Database	PostgreSQL stores synchronized reporting data plus application users, access mappings, sync history, and audit records.
4. Backend API	FastAPI handles authentication, hierarchy rules, database access, filters, report calculations, and export logic.
5. Web Portal	Next.js presents dashboards, records, reports, user registration, and administration.

6. Data Synchronization Strategy

The initial recommendation is automated scheduled synchronization rather than making every web request query Sage directly. This gives predictable performance and prevents reporting traffic from affecting the source ERP.

Track last successful synchronization time.

Track source system and synchronization status.

Track inserted/updated/failed record counts where applicable.

Support full and incremental synchronization strategies depending on available source keys and timestamps.

Display a simple user-facing message such as “Data accurate through: [date/time]”.

Allow synchronization frequency to increase later if the source platform supports it reliably.

7. Existing Sample Data Reviewed

The supplied ChubbStub.mdb extract contains three representative Sage tables used to validate the initial data shape and volume.

Sample Table	Records	Purpose / Observation
AR_InvoiceHistoryHeader	257,469	Invoice/billing-level information including customer, dates, amounts, salesperson/account references, and audit fields.
AR_InvoiceHistoryDetail	916,741	Invoice line-level benefit/product data including item codes, descriptions, quantities, amounts, tax and commission-related fields.
AR_Salesperson	209	Source master data that appears to contain organization/facility/account-style records in the supplied sample.

The final production model is expected to use approximately 10 source tables. Exact table relationships and business terminology will be confirmed during discovery.

8. Authentication & Hierarchical Access

8.1 End Users

End users may self-register using lightweight verification such as email and phone verification. Once their identity is matched to the appropriate source record, they receive access only to their own records.

8.2 Managerial / Administrative Users

Managerial and administrative access requires pre-approval. Access is hierarchical and must be enforced by the backend API, not only by hiding frontend navigation.

Access Level	Visibility
Location	Access only to one authorized location.
Client	Access to all authorized locations for that client.
Broker	Access to all authorized clients and their locations for that broker.
ALL	Internal access across all brokers, clients, locations, and records.

8.3 Example

If a broker serves McDonald’s and KFC, a McDonald’s location manager sees only that location; a McDonald’s corporate user sees all McDonald’s locations; the broker sees both McDonald’s and KFC; internal ALL users can see the complete hierarchy.

9. Proposed Application Modules

Module	Scope
Authentication & Registration	Sign in/out, self-registration, email/phone verification, password reset, session management.
Administrative Access Approval	Approve managerial users and assign Location, Client, Broker, or ALL visibility.
Dashboard	Hierarchy-aware totals, trends, counts, and filters.
Employee / Member Directory	Search, filtering, pagination, associated client/location, benefits, premium and billing history.
Employee / Member Detail	General profile, benefits/products, premiums, billing and historical records.
Broker / Client / Location Views	Dedicated hierarchy-aware views for the scope each user is authorized to see.
Invoice / Premium History	Invoice search, date and hierarchy filters, invoice header and line-level detail.
Reporting	Benefit, premium, client, location, broker, invoice, monthly/yearly and historical comparison reports.
Exports	CSV and Excel exports; PDF exports where specifically required.
Administration	Users, approvals, access mappings, sync history, configuration and audit activity.
Synchronization Monitoring	Last update, source, status, record counts, errors, and duration.

10. Scale & Performance

The platform should be designed for multi-million-record reporting from the beginning. The browser will not load full source tables. Search, filtering, aggregation, and pagination will be performed on the backend/database.

PostgreSQL indexes for commonly filtered/joined fields.

Server-side pagination and filtering.

Optimized joins and aggregation queries.

Background synchronization jobs.

Selective caching for expensive, repeatable reporting queries where appropriate.

Performance testing using production-like row counts.

11. Security & Audit

Secure password storage and session handling.

Email and phone verification for end-user registration.

Backend-enforced hierarchy authorization.

HTTPS and secure environment configuration.

Rate limiting and input validation.

Least-privilege database access.

Audit logging for approvals, access changes, login/security activity, and synchronization activity.

Managerial and administrative access requires pre-approval.

Any specific legal, regulatory, data-retention, or compliance requirements should be confirmed during discovery before final implementation.

12. Hosting & Branding

The application should appear as part of the company's existing web presence. The exact hosting model will depend on the capabilities of the current website environment.

Option	Approach
Existing hosting	Use the current infrastructure if it can reliably support Next.js, Python/FastAPI, PostgreSQL access, SSL, and scheduled/background jobs.
Dedicated application hosting	Host the portal separately while retaining full branding and connect it through a company subdomain such as portal.companydomain.com or reports.companydomain.com.

13. Recommended Development Phases

Phase	Estimated Effort	Primary Scope
Phase 1 - Discovery & Architecture	20-30 hrs	Production Sage table review, ~10-table mapping, reports, access hierarchy, user-record matching, synchronization strategy, PostgreSQL schema, hosting review.
Phase 2 - Application Foundation	45-55 hrs	Next.js, FastAPI, PostgreSQL, authentication, verification, hierarchical authorization, staging/development environments and base UI.
Phase 3 - Source Integration & Synchronization	60-80 hrs	Sage connector, source mappings, scheduled sync, validation, logging, incremental/full strategy and large-dataset optimization.
Phase 4 - Core Portal	100-115 hrs	Dashboard, directory/detail views, broker/client/location hierarchy, benefits, invoice/premium history, filters and pagination.
Phase 5 - Reports & Exports	45-65 hrs	Reporting APIs/pages, hierarchy-aware filters, aggregations, CSV/Excel and required PDF exports.

Phase 6 - Administration	35-45 hrs	User approvals, access assignments, hierarchy mappings, sync history, audit logs and settings.
Phase 7 - QA, Performance & Deployment	45-60 hrs	Automated tests, authorization tests, Sage reconciliation, performance testing, security checks, deployment and documentation.

14. Preliminary Development Estimate

Preliminary Total

350-450 hours

This is a planning range, not a fixed-price commitment. The estimate can be narrowed after the final Sage tables, source relationships, report definitions, calculations, synchronization method, and end-user matching rules are confirmed.

The largest remaining effort variables are report complexity, source-table relationships, data synchronization constraints, and the exact business rules required to map source records into the Broker → Client → Location hierarchy.

15. Primary Deliverables

Branded responsive Next.js reporting portal

FastAPI backend API

PostgreSQL reporting/application database

Sage data synchronization layer

Support for approximately 10 required production source tables

Hierarchical Location → Client → Broker → ALL authorization

End-user self-registration with email/phone verification

Managerial/admin pre-approval workflow

Dashboard and hierarchy-aware views

Employee/member, benefits, invoice and premium reporting

Required CSV/Excel exports and agreed PDF reports

Synchronization monitoring and “data accurate through” indication

Administrative portal and audit logging

Production deployment and technical documentation

16. Current Assumptions

Sage remains the authoritative source of business data.

The portal does not modify Sage business records.

Approximately 10 source tables provide the data required for the initial reporting solution.

The largest source table may grow to approximately 2 million records.

PostgreSQL is used as the optimized reporting/application database.

Daily automated synchronization is acceptable for the initial release unless discovery identifies a stronger real-time requirement.

End users access only their own matched records.

Managerial/administrative access is pre-approved and follows the agreed hierarchy.

The final report list, fields, calculations, and export requirements will be locked during discovery.

Separate application hosting is acceptable if the current website environment cannot support the selected stack, provided

branding/domain continuity is maintained.

17. Items to Confirm During Discovery

Identify the approximately 10 production Sage tables required for the reports.

Confirm the business meaning of the source identifiers and relationships.

Confirm how employees/members are uniquely identified and matched to self-registering users.

Confirm the exact Broker → Client → Location relationships in the source data.

Confirm mandatory Version 1 reports, fields, calculations, filters, and exports.

Confirm available Sage connectivity (ODBC, SQL Server, scheduled files, or other supported method).

Confirm whether daily synchronization is sufficient for Version 1.

Confirm the email/phone verification provider and matching process.

Confirm current website hosting capabilities and preferred domain/subdomain arrangement.